ECE 212 / EXAM REFERENCE / ARM + SYSTEMS

ECE 212 Exam Cheatsheet Book

Consolidated from OCR notes in Parts E, F, and G: assembly language deep dive, stacks and subroutines, and hardware interfacing/address decoding. This edition cleans OCR damage, restores missing notation where intent is clear, and adds compact explanations, worked examples, traps, and solve patterns for exam use.

ARMv7-M / Thumb-2 | Stacks + AAPCS | DMA + Decoding | Worked Examples | Exam Traps

Built for open-book exam speed: start with the coverage map, then jump to the exact table, example, or branch/stack/decoder pattern you need.

How to use this book: skim the section banner, find the relevant instruction family or hardware topic, read the bolded trap or solve pattern, and only drop into the worked example if you need the full reasoning chain.

INCLUDED

- All named topics from all three OCR text files
- OCR cleanup and notation repair
- Short explanations instead of raw opcode dumps
- Subroutine and stack conventions
- Address-decoder design patterns
- Practice-style “common mistakes” sections

Part E

Data movement, arithmetic, logic, flags, shifts, branches, IT blocks, multiply/divide, saturation, extension, bit fields.

Part F

Full descending stack, BL/BX/LR, register save-restore, AAPCS, parameter passing, modular program design.

Part G

Memory-mapped peripherals, data bus vs port size, timing diagrams, DMA, full/partial/mixed-mode decoding.

PREPARED FROM OCR TEXT IN THE CURRENT FOLDER ON 2026-03-31

Table of Contents

Page numbers refer to the PDF edition of this book.

Section	Page
Table of Contents	**2**
Part E: ARMv7-M Assembly Deep Dive	**3**
Quick Register and Flag Review	3
E.3 Data-Movement Operations	3
E.4 Data Processing Operations I	4
E.5 Comparison and Sequence Control	5
E.6 Data Processing Operations II	7
Part F: Stacks, Subroutines, and Program Design	**9**
F.4 The Stack	9
F.5 Subroutines	10
F.5.7 Passing Parameters	11
F.6 Program Design Process	12
Part G: Hardware Interfacing, DMA, and Address Decoding	**13**
G.4 Memory-Mapped Devices	13
G.5 Memory-to-CPU Interface	13
G.6 Timing Diagrams	14
G.7 Direct Memory Access (DMA)	14
G.8 Address Decoding	14
Practice Sections	**17**
Final Review Checklist	17

All OCR-derived topics are still included below in the body of the reference; this contents page is intentionally compressed so you can jump to the right PDF page quickly during review.

Part E: ARMv7-M Assembly Deep Dive

E.1-E.2 PURPOSE AND LEARNING GOALS

- Goal: build enough working ARMv7-M / Thumb-2 literacy to read instructions, validate them, and infer effects on registers, memory, and APSR flags.
- Major ideas: data movement, arithmetic and logic, comparisons, shifts, branching, IT blocks, multiplication/division, extension, bit fields, and system/status moves.
- Exam standard: know what instruction forms are legal, what gets written, how effective addresses are formed, and which flags matter.

QUICK REGISTER AND FLAG REVIEW

- General-purpose registers: `R0–R12`
- `R13 = SP`, `R14 = LR`, `R15 = PC`
- Word = 32 bits, halfword = 16 bits, byte = 8 bits
- Little-endian memory for Cortex-M: lowest-addressed byte is least significant byte of the word
- `N`: negative result
- `Z`: result is zero
- `C`: carry out / no borrow
- `V`: signed overflow
- `Q`: saturation occurred

E.3 DATA-MOVEMENT OPERATIONS

E.3.1 Loading a Fixed Value into a Register

Instruction	Use	Key Idea	Notes
`MOVS Rd, #imm8`	Small immediate	Zero-extends 8-bit immediate into `Rd`	Common 16-bit form, often outside IT block
`MOV{S}.W Rd, #imm`	Larger modified immediate	Uses Thumb expanded immediate encoding	Not every 32-bit constant is directly encodable
`MOVW Rd, #imm16`	Load low 16 bits	Loads a zero-extended 16-bit immediate into `Rd`	Use with `MOVT` for arbitrary 32-bit values
`MOVT Rd, #imm16`	Load top 16 bits	Writes `Rd[31:16] = imm16` and leaves `Rd[15:0]` unchanged	Usually do `MOVW` first, then `MOVT`
`MVN Rd, #imm`	Move bitwise NOT of immediate	`Rd = ~imm32`	Useful when the complement is encodable but the original constant is not
`LDR Rd, =const`	Pseudo-instruction	Assembler chooses best way to materialize constant	May become `MOV`, `MVN`, or literal-pool load
`MOV32 Rd, #imm32`	Pseudo-instruction	Assembler usually emits `MOVW` + `MOVT`	Good conceptual shortcut

Arbitrary 32-bit constant pattern:
```
MOVW Rd, #low16
MOVT Rd, #high16
```
Do not swap them. If you do `MOVT` first then `MOVW`, the low-half write can destroy part of the value construction you intended.

E.3.2 Loading an Address into a Register

- `ADR Rd, label` computes a PC-relative address.
- Assembler turns it into an `ADD` or `SUB` relative to aligned `PC`.
- Use it when you want the *address* of a label, not the *data stored at* that label.
- Typical confusion: `ADR R0, table` gives pointer to `table`; `LDR R0, table` often means load a value from memory near `PC`.

E.3.3-E.3.4 Register Moves and Shifted Register Moves

- `MOV Rd, Rm`: copy one register to another.
- `MOV Rd, Rm, LSL #k` and related forms exist, but course notes prefer using standalone `LSL/LSR/ASR/ROR/RRX`.
- Barrel shifter = one operand can be shifted/rotated as part of the instruction datapath.

E.3.5-E.3.6 LOADS, STORES, AND ADDRESSING MODES

Family	Meaning	Size / Extension

LDR, STR	Load/store word	32-bit
LDRB, STRB	Load/store byte	8-bit, zero-extended on load
LDRH, STRH	Load/store halfword	16-bit, zero-extended on load
LDRSB	Load signed byte	8-bit sign-extended to 32 bits
LDRSH	Load signed halfword	16-bit sign-extended to 32 bits
LDRD, STRD	Dual-register load/store	64-bit via two registers
LDM, STM	Load/store multiple registers	Bulk transfer

Addressing Forms

Form	Syntax	Effective Address	Write-back?
Indirect	`[Rn]`	`EA = Rn`	No
Offset	`[Rn, #imm]` or `[Rn, Rm]`	`EA = Rn + offset`	No
Pre-indexed	`[Rn, #imm]!`	`Rn = Rn + offset; EA = Rn`	Yes
Post-indexed	`[Rn], #imm`	`EA = Rn; Rn = Rn + offset`	Yes

- Literal load: `LDR Rt, label` or effectively `LDR Rt, [PC, #imm]`.
- Register offset: `LDR Rt, [Rn, Rm]`.
- Scaled register offset: `LDR.W Rt, [Rn, Rm, LSL #imm2]`.
- Word accesses should be word-aligned; halfwords should be halfword-aligned.
- SP-based addressing is common for locals, parameters, and saved registers.

Worked Example: Decode the Addressing Effect

```
Suppose R6 = 0x11223300.

LDR  R1, [R6]         ; use address 0x11223300, R6 unchanged
LDR  R2, [R6, #4]     ; use address 0x11223304, R6 unchanged
LDR  R3, [R6, #-6]    ; use address 0x112232FA, R6 unchanged
LDR  R4, [R6, #-6]!   ; first R6 = 0x112232FA, then load from that address
LDR  R5, [R6], #8     ; load from old R6, then R6 = R6 + 8

Exam rule:
1. Compute effective address first from syntax.
2. Decide transfer size and sign/zero extension.
3. Apply write-back only if '!' or post-index form is present.
```

E.3.7 MISCELLANEOUS DATA MOVEMENT

Instruction	Effect	Typical Use
`RBIT Rd, Rm`	Reverse all 32 bits	Bit-order transforms
`REV Rd, Rm`	Reverse bytes in a word	Endianness conversion
`REV16 Rd, Rm`	Reverse bytes within each halfword	Packed halfword data
`REVSH Rd, Rm`	Reverse lower halfword bytes then sign-extend	Signed 16-bit byte-swapped input

E.4 DATA PROCESSING OPERATIONS I

E.4.1 Basic Arithmetic

Instruction	Meaning	Model
`ADD`	Add	`A + B`
`ADC`	Add with carry	`A + B + C`
`SUB`	Subtract	`A – B`
`SBC`	Subtract with carry = subtract with borrow support	`A – B – (1–C)`
`RSB`	Reverse subtract	`B – A`

- ARM unifies these through the internal idea of `AddWithCarry(A, B, Cin)`.
- Extended-precision addition/subtraction depends on carrying `C` from one word to the next.
- Negation pattern: `RSB Rd, Rn, #0` gives `Rd = 0 - Rn = -Rn`.

E.4.2 Basic Logical Operations

Instruction	Meaning	Bit-mask Use
`AND`	Bitwise AND	Clear bits where mask has 0
`BIC`	Bit clear = `A AND (~B)`	Clear selected bits directly
`ORR`	Bitwise OR	Set bits where mask has 1
`ORN`	`A OR (~B)`	Less common complement form
`EOR`	Bitwise XOR	Toggle bits where mask has 1
`MVN`	Bitwise NOT of operand	Create complement

E.4.3 Comparison and Test

Instruction	Hidden Operation	Result Stored?	Flags
`CMP A, B`	`A - B`	No	N, Z, C, V
`CMN A, B`	`A + B`	No	N, Z, C, V
`TST A, B`	`A AND B`	No	N, Z, C
`TEQ A, B`	`A XOR B`	No	N, Z, C

E.4.4 Shifts and Bit Manipulation

Instruction	Meaning	Useful Interpretation
`LSL`	Logical shift left	Multiply unsigned value by `2^k` if no overflow matters
`LSR`	Logical shift right	Unsigned divide by `2^k`
`ASR`	Arithmetic shift right	Signed divide by `2^k` with sign replication
`ROR`	Rotate right	Bit cycle; no information discarded
`RRX`	Rotate right through carry	Carry becomes new MSB

E.5 COMPARISON AND SEQUENCE CONTROL

E.5.1, E.5.3 Branching Instructions

Instruction	Meaning	Typical Use
`B label`	Unconditional branch	Loop, skip, jump
`Bcc label`	Conditional branch	If/while/for logic
`BL label`	Branch with link	Call subroutine; saves return address in LR
`BX Rm`	Branch to address in register	Commonly `BX LR` to return
`BLX Rm`	Branch with link to address in register	Indirect function call

E.5.2 Condition Codes

Code	Meaning	Flag Test	Interpretation
`EQ`	Equal	`Z=1`	Last result was zero / operands equal
`NE`	Not equal	`Z=0`	Operands differ
`CS/HS`	Carry set / unsigned higher or same	`C=1`	Unsigned compare
`CC/LO`	Carry clear / unsigned lower	`C=0`	Unsigned compare
`MI`	Minus	`N=1`	Negative

PL	Plus	N=0	Non-negative
VS	Overflow	V=1	Signed overflow happened
VC	No overflow	V=0	No signed overflow
HI	Unsigned higher	C=1 and Z=0	> for unsigned
LS	Unsigned lower or same	C=0 or Z=1	<= for unsigned
GE	Signed greater or equal	N=V	>= for signed
LT	Signed less than	N!=V	< for signed
GT	Signed greater than	Z=0 and N=V	> for signed
LE	Signed less or equal	Z=1 or N!=V	<= for signed
AL	Always	True	Unconditional

Unsigned vs signed exam trap: after `CMP`, use `HI/HS/LO/LS` for unsigned comparisons and `GT/GE/LT/LE` for signed comparisons.

E.5.4 Compare and Branch

- `CBZ Rn, label`: branch if `Rn == 0`
- `CBNZ Rn, label`: branch if `Rn != 0`
- Equivalent idea to `CMP Rn, #0` then `BEQ/BNE`, but does not modify APSR flags.
- Useful for loop exit and null/zero tests.
- Use them when you only care about zero/nonzero and want later branches to still see the old APSR flags.
- These are PC-relative branches and are best thought of as short local tests, not general replacements for every compare-then-branch sequence.

E.5.5 Conditional IT Block

- `IT{x{y{z}}} cc` makes the next 1 to 4 instructions conditional.
- `T` means use base condition; `E` means use inverse condition.
- Example: `ITTE GT` means instruction 1 = GT, instruction 2 = GT, instruction 3 = LE.
- IT blocks may not contain nested IT blocks.
- Instructions that modify `PC` must be outside the block or the last instruction in the block.
- `CBZ/CBNZ` may not be inside an IT block.
- `BKPT` executes even inside an IT block.

Inside an IT block	Status	Reason / Reminder
Ordinary data-processing instruction	Usually allowed	Normal use case for conditional execution
Nested `IT`	Not allowed	No nesting
`CBZ`, `CBNZ`	Not allowed	Must be outside the IT block
Instruction writing `PC`	Last only	Branch/return style instructions must end the block
`BKPT`	Executes anyway	Breakpoint is not suppressed by the IT condition

```
Example:
CMP   R0, #9
ITE   GT
ANDGT R0, R0, R1
MOVLE R2, R3

Meaning:
if (R0 > 9 signed) then
    R0 = R0 AND R1
else
    R2 = R3
```

WORKED EXAMPLE: CAPITALIZE AN ASCII STRING

The OCR notes start with a programming problem asking for a null-terminated string copy that capitalizes lowercase letters.

```
; R0 = input pointer, R1 = output pointer
loop:
    LDRB  R2, [R0], #1      ; get next char, advance input
    CBZ   R2, done          ; if NUL, finish
    CMP   R2, #0x61         ; 'a'
    BLO   skip
    CMP   R2, #0x7A         ; 'z'
    BHI   skip
    SUB   R2, R2, #0x20     ; lowercase -> uppercase
skip:
    STRB  R2, [R1], #1      ; write char, advance output
    B     loop
done:
    STRB  R2, [R1]          ; store final NUL
```

Why it works: lowercase ASCII letters occupy a contiguous block from `0x61` to `0x7A`; uppercase versions are exactly `0x20` smaller.

E.6 DATA PROCESSING OPERATIONS II

E.6.1-E.6.2 Multiplication

Instruction	Operation	Notes
`MUL`	`(32-bit) * (32-bit) -> low 32 bits`	Signed/unsigned same for low 32 bits
`MLA`	`Ra + Rn*Rm`	Multiply accumulate
`MLS`	`Ra - Rn*Rm`	Multiply subtract
`SMULL`	Signed 32x32 -> 64 bits	Stores low/high in two registers
`UMULL`	Unsigned 32x32 -> 64 bits	Use when interpretation matters
`SMLAL`	Signed 64-bit accumulate	Adds product into existing 64-bit value
`UMLAL`	Unsigned 64-bit accumulate	Unsigned version

E.6.3 Division

- `SDIV Rd, Rn, Rm`: signed integer division, rounds toward zero.
- `UDIV Rd, Rn, Rm`: unsigned integer division.
- These are integer operations, not floating-point division.

E.6.4 Saturating

- `SSAT Rd, #s, Rn {, shift}`: clamp result into signed `s`-bit range `[-2^(s-1), 2^(s-1)-1]`.
- `USAT Rd, #s, Rn {, shift}`: clamp result into unsigned `s`-bit range `[0, 2^s-1]`.
- If clamping occurs, set `APSR.Q`.

E.6.5 Sign/Zero Extension

Instruction	Meaning	Result
`UXTB`	Unsigned extend byte	low 8 bits -> 32 bits, zero-filled
`UXTH`	Unsigned extend halfword	low 16 bits -> 32 bits, zero-filled
`SXTB`	Signed extend byte	low 8 bits -> 32 bits, sign-filled
`SXTH`	Signed extend halfword	low 16 bits -> 32 bits, sign-filled

Optional rotate before extension lets you pick another byte/halfword position from the source register.

E.6.6 Bit Field Operations

Instruction	Use
`BFC Rd, #lsb, #width`	Clear selected bit field in destination
`BFI Rd, Rn, #lsb, #width`	Insert low bits of source into destination field
`CLZ Rd, Rm`	Count leading zeros
`SBFX Rd, Rn, #lsb, #width`	Extract field then sign-extend

`UBFX Rd, Rn, #lsb, #width`	Extract field then zero-extend

E.6.7 Extended-Length Arithmetic

For numbers wider than 32 bits, do the least significant word first, then ripple the carry/borrow upward.

```
; 64-bit addition: A = A_hi:A_lo in R5:R4, B = B_hi:B_lo in R1:R0
ADDS  R4, R4, R0
ADCS  R5, R5, R1

; 96-bit pattern:
ADDS  low,  low,  low2
ADCS  mid,  mid,  mid2
ADCS  high, high, high2
```

E.6.8 Boolean Operations with Masks

- Clear selected bits: `AND Rd, Rn, #mask` where mask has 0s in the bits to clear.
- Set selected bits: `ORR Rd, Rn, #mask` where mask has 1s in the bits to set.
- Toggle selected bits: `EOR Rd, Rn, #mask` where mask has 1s in the bits to toggle.
- Bitwise NOT pattern: `MVN Rd, Rm` or equivalent OR/AND combinations when needed.

E.6.9 Miscellaneous Operations

Instruction	Meaning	Exam Note
`BKPT #imm8`	Breakpoint for debugging	Immediate encodes but has no arithmetic meaning
`MRS Rd, APSR`	Move from special register	Read status/control style register
`MSR APSR, Rn`	Move to special register	Write status/control style register

PART E EXAM TRAPS AND FAST CHECKS

- **Address vs data:** `ADR` gets address, `LDR` usually gets data.
- **Signed vs unsigned branch:** use the right condition family.
- **Post-index vs pre-index:** write-back timing changes the address used.
- **LDRB/LDRH vs LDRSB/LDRSH:** zero-extension and sign-extension are not interchangeable.
- **IT blocks:** no nested IT, no `CBZ/CBNZ`, and PC-changing instructions only last.
- **Carry in subtraction:** ARM `C=1` means no borrow.
- **Extended-length arithmetic:** use `ADDS` or `SUBS` first, then `ADCS` or `SBCS` upward.
- **Literal loads:** PC-relative range and alignment matter.

Part F: Stacks, Subroutines, and Program Design

F.1-F.3 PURPOSE, MOTIVATION, AND CORE IDEA

- Need temporary storage because registers are limited.
- Need support for subroutines and exception handling.
- Need a simple way to track used temporary memory with one pointer.
- That data structure is the stack.

F.4 THE STACK

F.4.1-F.4.2 Basic Definitions and Terminology

- Stack = FILO / LIFO structure: first in, last out.
- **Push**: store new item on top of stack.
- **Pop**: retrieve top item and remove it from stack.
- **Top of stack**: item that would be popped next.
- **Stack pointer (SP)**: register tracking top / next free location depending on convention.
- **Stack origin**: initial boundary of reserved stack region.

F.4.3 Stack Types

Choice	Option A	Option B
Growth direction	Descending: grows toward lower addresses	Ascending: grows toward higher addresses
SP meaning	Full: SP points at top item	Empty: SP points at next free slot

ARM convention in this course: **full descending stack**.

F.4.4 Stack Implementation

- Reserved contiguous memory block used only for stack.
- `SP = R13`.
- SP must be word-aligned; under AAPCS, effectively 8-byte aligned at call boundaries.
- Push on full descending stack = pre-decrement SP, then store.
- Pop on full descending stack = load from SP, then post-increment SP.

```
; Example stack region
; stack occupies 0x00010000 to 0x00012FFF
MOV   SP, #0x13000      ; stack origin just above top

; push R0
STR   R0, [SP, #-4]!

; pop into R0
LDR   R0, [SP], #4
```

F.4.5-F.4.6 Primary Uses of the Stack

- Save return information during calls.
- Save and later restore register values.
- Pass parameters and return values when registers are insufficient.
- Subroutine rule: if you push `k` words, you must pop or otherwise reclaim the same `k` words before returning, unless the calling convention explicitly says otherwise.

Stack Frame and Frame Pointer

- A **stack frame** is the chunk of stack memory belonging to one active subroutine call.
- Typical contents: saved `LR`, saved callee-saved registers, local variables, and sometimes stack-passed parameters.
- If `SP` changes during the routine, offsets from `SP` can become hard to track.
- A **frame pointer** is a register copied from `SP` at a stable point so locals and parameters can be referenced with fixed offsets.
- Exam use: if the problem asks for a stable base for locals/params after several pushes or local allocations, use a frame pointer.

```
; One common pattern
PUSH {R4-R7, LR}       ; save callee-saved regs + return address
SUB  SP, SP, #16       ; reserve locals
MOV  R7, SP            ; R7 used here as frame pointer

; examples of fixed-offset access
LDR  R0, [R7, #0]      ; local 0
LDR  R1, [R7, #4]      ; local 1
LDR  R2, [R7, #16]     ; first stacked item above locals

MOV  SP, R7            ; discard locals
ADD  SP, SP, #16
POP  {R4-R7, PC}
```

Frame-pointer exam rule: if the callee keeps pushing, popping, or allocating temporary space, offsets from SP move around; offsets from a frame pointer stay stable.

F.5 SUBROUTINES

F.5.1 Basics

- Subroutine = reusable program segment callable from elsewhere.
- Benefits: reuse, smaller code size, modularity, cleaner design.
- Cost: call/return overhead compared with inline code.

F.5.2 Calling a Subroutine

- `BL label`: branch with link; writes return address into LR.
- `BLX Rm`: indirect call through register; also writes return address into LR.
- Leaf subroutine = does not call another subroutine.
- Non-leaf subroutine = calls another subroutine, so it must preserve its own caller's return address first.

F.5.3 Returning from a Subroutine

Pattern	Use
`BX LR`	Standard return
`MOV PC, LR`	Also works with restrictions
`POP {R4-R6, PC}`	Restore regs and return in one instruction

```
; Leaf subroutine
subr:
    ; use only caller-saved regs if possible
    BX   LR

; Non-leaf subroutine
subr:
    PUSH {R4-R6, LR}
    ; body
    BL   helper
    ; more body
    POP  {R4-R6, PC}
```

F.5.4 Additional Concepts

- **Position-independent code**: avoids absolute addresses so code works regardless of load location.
- **Re-entrant**: multiple active invocations can coexist safely.
- **Transparent**: subroutine restores registers it was not supposed to change.
- All of these push you toward a formal calling convention.

F.5.5-F.5.6 Saving and Restoring Multiple Registers

Operation	General Form	Stack-Friendly Alias
Store multiple	`STMDB SP!, {regs}`	`PUSH {regs}`
Load multiple	`LDMIA SP!, {regs}`	`POP {regs}`

- Lowest-numbered register maps to lowest memory address in the block transfer.
- For a single register save/restore, `STR [SP, #-4]!` and `LDR [SP], #4` are fine.
- Do not casually include `SP` or `PC` in the wrong lists.

F.5.7 PASSING PARAMETERS

Method	Pass by Value	Pass by Reference	Pros	Cons
Registers	Put data in regs	Put address in regs	Fast, re-entrant, position-independent	Limited quantity, consumes working registers
Memory location	Agreed fixed memory slots	Address known to both caller/callee	Simple conceptually	Not PIC-friendly, not re-entrant, shared-state risk
Stack	Push raw values	Push addresses	Scales to many params, flexible	More memory traffic and stack bookkeeping

AAPCS Highlights

- Stack is full descending.
- SP must be aligned, and call boundaries should preserve 8-byte alignment.
- Arguments 1-4 in `R0-R3`.
- Return values 1-4 in `R0-R3`.
- `R0-R3` and `R12` are caller-saved / scratch.
- `R4-R11` are callee-saved; restore them if you modify them.
- Callee must preserve SP.

> **Performance conclusion from the notes:** register passing is the preferred AAPCS-compliant path because it minimizes stack traffic and can eliminate memory access entirely for simple subroutines.

Pass by Value vs Pass by Reference

Style	What You Pass	Callee Sees	When to Prefer It
Pass by value	The actual data bits	A copy of the input	Small scalar inputs that the callee should not modify in caller memory
Pass by reference	An address / pointer	Where the caller's data lives	Large objects, output parameters, or data that must be updated in place

WORKED EXAMPLE: 64-BIT ADD ON THE STACK

Idea from the notes: inputs are two 64-bit values, and the subroutine overwrites one stacked operand with the result.

```
; Caller pushes PL, PH, QL, QH in registers first, then:
PUSH {R0-R3}
BL   add64
POP  {R4, R5}      ; get RL, RH
ADD  SP, SP, #8    ; discard remaining stacked words if convention requires

add64:
    PUSH {R6-R9}
    LDR  R6, [SP, #16]   ; PL
    LDR  R7, [SP, #20]   ; PH
    LDR  R8, [SP, #24]   ; QL
    LDR  R9, [SP, #28]   ; QH
    ADDS R6, R6, R8      ; RL
    ADCS R7, R7, R9      ; RH
    STR  R6, [SP, #16]
    STR  R7, [SP, #20]
    POP  {R6-R9}
    BX   LR
```

Key exam lesson: stack offsets must be computed after accounting for the callee's own pushes.

WORKED EXAMPLE: AAPCS-COMPLIANT 64-BIT ADD

```
; Inputs:
; R0 = PL, R1 = PH, R2 = QL, R3 = QH
; Outputs:
; R0 = RL, R1 = RH

add64_aapcs:
    ADDS R0, R0, R2
    ADCS R1, R1, R3
    BX   LR
```

This is the ideal exam answer when the problem explicitly allows AAPCS register passing.

F.6 PROGRAM DESIGN PROCESS

Topic	Meaning	Why It Matters in Assembly
Top-down design	Break big problem into smaller tasks	Keeps low-level coding manageable
Modular design	Single-purpose modules with defined interfaces	Independent testing and replacement
Structured programming	Build from sequence, decision, and loop constructs	Reduces spaghetti control flow
Testability	Ease of verifying operation	Supports debugging with breakpoints and module tests
Recoverability	Ability to cope with errors	Important for robust routines and exception handling

Control Structure Mapping

- **IF-THEN**: compare/test, then conditional branch over action if false.
- **IF-THEN-ELSE**: conditional branch to ELSE, execute THEN, unconditional branch to END, then ELSE block.
- **WHILE**: test at top, branch out if false.
- **DO-WHILE**: body first, test at bottom, branch back if true.
- **FOR**: initialize, top test, body, increment, branch back.
- **SELECT/CASE**: compare against cases and branch to matching handler or default.

F.7 Program Design Language (PDL)

- Pseudocode is the bridge between high-level reasoning and assembly implementation.
- Describe what needs to happen before deciding exact registers/opcodes.
- Keep statements English-like but precise enough that coding becomes almost mechanical.
- Good PDL later turns into comments, tests, reviews, and ports.

PART F EXAM TRAPS AND SOLVE PATTERNS

- **Full descending stack:** push = pre-decrement, pop = post-increment.
- **Non-leaf subroutine:** save LR before calling another routine.
- **Caller-saved vs callee-saved:** know which side is responsible.
- **SP offsets:** if you push registers in the callee, all parameter offsets move.
- **Return cleanup:** know whether caller or callee removes stack arguments.
- **Transparency:** if not returning a value in a register, restore it.
- **Stack discipline:** never read above the top of stack; do not rely on stale data.

Part G: Hardware Interfacing, DMA, and Address Decoding

G.1-G.3 BIG PICTURE AND BACKGROUND

- The CPU talks to memory and peripherals through address, data, and control signals.
- Many devices are memory-mapped, so the CPU accesses them using addresses just like memory.
- The STM32L432KC integrates many on-chip peripherals: GPIO, DMA, ADC/DAC, SPI, I2C, USART, timers, CAN, USB, and more.
- GPIO ports use control/configuration registers plus input/output data registers.

Signal Vocabulary

Term	Meaning
Active-high	Signal is true when logic high
Active-low	Signal is true when logic low
Asserted	Driven to active state
Negated	Driven to inactive state
Tri-state	Can be 0, 1, or high-impedance / floating

GPIO Register View

Register Type	Purpose
Mode / configuration register	Choose input, output, analog, or alternate-function behavior
Input data register	Read current logic levels on pins configured as inputs
Output data register	Drive logic levels on output pins
Alternate-function selection	Hand pin control to a peripheral such as SPI, I2C, USART, or a timer

G.4 MEMORY-MAPPED DEVICES

- External devices can share the processor buses and occupy regions of the address space.
- Examples: RAM, ROM, LCD controller, I/O peripherals.
- On-chip peripherals are also memory-mapped inside the MCU.
- Only one target device should be active for a given bus transaction.

G.5 MEMORY-TO-CPU INTERFACE

Bus and Port Size

- Cortex-M4 here is modeled with a 32-bit address bus and 32-bit data bus.
- Memory capacity is counted in bytes, but memory chips may have 8-bit, 16-bit, or 32-bit ports.
- A 32-bit CPU talking to narrower memory may require multiple transfers for one word access.

Memory Port Size	Byte Access	Halfword Access	Word Access
8-bit	1 transfer	2 transfers	4 transfers
16-bit	1 transfer	1 transfer	2 transfers
32-bit	1 transfer	1 transfer	1 transfer

Byte Enable / Byte Select Lines

- CPU has byte-enable lines to indicate which byte lanes of the data bus are active.
- With a 32-bit bus, the active byte lane depends on `A1:A0` and transfer size.
- Misalignment complicates transfers and can require multiple cycles or be disallowed depending on architecture/peripheral.

```
Byte write to address 0x1234 on a 32-bit port:
1. Address lines carry 0x00001234
2. Low bits A1:A0 = 00 select the lowest byte lane
3. Only the corresponding byte-enable line is asserted
4. Data appears on the matching byte lane of D[31:0]
```

Exam question pattern: given address, transfer size, and memory port width, determine: address bits, active byte lanes, number of transfers, and whether the access is aligned.

Worked Example: Misaligned Word on a 32-bit Port

```
Question:
Can a 32-bit word be stored starting at address 0x0003 on a 32-bit-wide port?

Reasoning:
1. A word needs 4 bytes.
2. Address 0x0003 starts in the highest byte lane of the first aligned 32-bit word.
3. Only byte 3 fits in the aligned word at addresses 0x0000-0x0003.
4. The remaining bytes must go into the next aligned word at addresses 0x0004-0x0006.

So the transfer spans two aligned bus words:
- first bus word contributes 1 byte
- second bus word contributes 3 bytes

Exam conclusion:
misaligned accesses break the clean one-transfer model. On a real target they may
require multiple transfers, extra hardware support, or may be disallowed.
```

G.6 TIMING DIAGRAMS

- Event sequence diagrams show causal ordering between signals.
- Data-sheet timing diagrams quantify delays and legal intervals.
- For sequential elements like D flip-flops, know setup time, hold time, and propagation delay.

Term	Meaning
Setup time	Data must be stable for this long before clock edge
Hold time	Data must remain stable for this long after clock edge
Propagation delay	Delay from input event to output change

Shaded/uncertain regions in timing diagrams usually mean the signal is not guaranteed valid there.

G.7 DIRECT MEMORY ACCESS (DMA)

- Normally CPU is bus master and executes instructions to move data.
- DMA lets a device transfer directly to/from memory without CPU performing each data move.
- CPU temporarily yields bus-master role to the DMA controller.

DMA Process

1. CPU starts as bus master.
2. A device requests bus use.
3. DMA controller gains control and performs memory transfers directly.
4. DMA releases bus and CPU resumes normal control.

Advantages and Modes

Advantage	Why
Speed	No fetch/decode/execute overhead for each moved word
Efficiency	CPU is freed for other work
Bus utilization	May use otherwise idle cycles

- **Cycle stealing**: DMA takes bus for one transfer at a time, interleaving with CPU.
- **Burst mode**: DMA performs consecutive transfers before releasing bus.

G.8 ADDRESS DECODING

Core Idea

- Address space is large; physical devices occupy selected subranges inside it.
- Decoder logic asserts the correct chip select (`CS`) based on address bus values.
- Only the selected device should respond to the bus cycle.

General Design Math

- If a chip stores `2^M` bytes, it needs `M` low-order address lines internally.
- If there are `2^N` equal-size blocks, use `N` high-order bits to choose block / chip.
- Total address bits = `N + M`.

Full vs Partial vs Block vs Mixed-Mode Decoding

Scheme	Meaning	Result
Full decoding	Use all relevant address lines	Each physical location has one unique logical address
Partial decoding	Ignore some address lines	Device appears at multiple mirrored addresses
Block decoding	Split address space into blocks using high-order bits	Simple decoder-driven memory map
Mixed-mode decoding	Combine full, partial, and/or block approaches	Trades simplicity against uniqueness/flexibility

Partial Decoding Consequences

- Undecoded lines become "don't cares".
- Same physical device appears multiple times in logical address space.
- Advantage: simpler, cheaper, faster decoder logic.
- Disadvantage: address aliasing / mirroring can be undesirable.

Design Procedure

1. Write target base address and device size.
2. Determine how many low bits belong to in-device addressing.
3. Mark the remaining high bits that distinguish devices.
4. For full decoding, constrain every necessary distinguishing bit.
5. For partial decoding, intentionally leave some bits as don't-care if aliasing is acceptable.
6. Generate a `CS` expression that asserts only for the chosen address pattern.

Rule from the Notes for Partial Decoding

> For any pair of devices, there should be a common address line used to generate their chip selects such that one device sees 0 on that line and the other sees 1.

Memory Map Design Considerations

- Device regions are usually continuous.
- Regions do not need to be adjacent.
- Some devices may only map to even or odd addresses depending on width.
- Decoder logic must be fast because every memory access depends on it.
- Leave room for future system changes when choosing map layout.

WORKED EXAMPLE: FULL VS PARTIAL DECODER THINKING

```
Example idea:
CPU address bus = 24 bits
ROM size = 64 kB = 2^16 bytes
Base address = 0x480000

1. Low 16 address bits select location within ROM.
2. High 8 bits must match 0x48 for full decoding.
3. So ROM range is 0x480000 to 0x48FFFF.
4. Full decoder asserts CS when A23:A16 = 0x48.

If you intentionally ignore some high bits, the same ROM would appear at
multiple mirrored ranges instead of one unique range.
```

WORKED EXAMPLE: MULTI-DEVICE PARTIAL DECODER

```
Example idea:
CPU address bus = 24 bits
Devices:
- ROM1 = 32 kB at highest logical addresses
- ROM2 = 32 kB just below ROM1
- RAM  = 128 kB at base address 0x000000
- PER1 = 8 registers
- PER2 = 8 registers

Step 1: low-order address bits used inside each device
- 32 kB device -> 15 low bits
- 128 kB device -> 17 low bits
- 8 byte registers -> 3 low bits

Step 2: choose a few high bits for chip select
- RAM near 0x000000 suggests top bits near 0
- high ROMs suggest top bits near 1
- peripherals should differ on at least one common decoded address line

Step 3: obey the partial-decoding rule
For any pair of devices, include at least one decoded address line where one device
expects 0 and the other expects 1.

Step 4: accept aliasing deliberately
Any high-order lines not used in the decoder become don't-cares, so each device may
appear in multiple mirrored logical regions.

What to write on an exam:
1. State how many low bits each device consumes internally.
2. Mark which high bits you decode for each chip select.
3. Use X or don't-care for intentionally ignored bits.
4. State that multiple logical copies exist for partially decoded devices.
```

Mixed-mode decoding idea: fully decode the devices that must have unique addresses, and partially decode the ones where simpler logic and aliasing are acceptable.

PART G EXAM TRAPS AND SOLVE PATTERNS

- **Memory-mapped device:** CPU talks to device by addressing it, not by some magical separate instruction family.
- **Port width vs bus width:** number of transfers changes with port size.
- **Byte enables:** do not forget lane selection when transferring bytes or halfwords on a 32-bit data bus.
- **Misalignment:** if the access starts at the wrong byte offset, count how many aligned bus words it spans.
- **Timing diagrams:** setup and hold are around the clock edge; propagation is after it.
- **DMA:** CPU is not moving every word; DMA controller is bus master during the transfer window.
- **Partial decoding:** aliasing is expected, not an error, if undecoded bits are left as don't-care.
- **Decoder math:** low bits = address within chip, high bits = chip selection.

Practice Sections

COMMON MISTAKES

- Using signed branch conditions for unsigned quantities after `CMP`.
- Forgetting that `LDRSB` and `LDRSH` sign-extend.
- Treating `ADR` like a memory read instead of address formation.
- Writing stack-frame offsets as if the callee had not pushed its own registers.
- Returning from a non-leaf subroutine without preserving `LR`.
- Ignoring byte-enable behavior on narrow or offset memory accesses.
- Designing a partial decoder without checking whether two devices can ever share the same asserted pattern.

FAST SOLVE PATTERNS

Problem Type	Fast Method
Decode branch meaning	Identify flags set immediately before it, then interpret signed vs unsigned condition family
Compute effective address	Parse bracket syntax, compute EA, then apply write-back timing
Wide arithmetic	Start from least significant word with `ADDS/SUBS`, propagate with `ADCS/SBCS`
Stack frame tracing	Draw SP after every push/pop, then label offsets from current SP
AAPCS question	Check R0-R3 first, then stack; preserve R4-R11 if callee touches them
DMA conceptual question	Emphasize bus-master transfer without CPU executing per-word moves
Address decoder	Device size determines low bits, base address determines required high bits

MINI WORKED PROBLEMS

1. When should you use `CBZ` instead of `CMP` + `BEQ`?

Use `CBZ` when you only need a zero/nonzero decision and you do not want to disturb APSR flags.

2. Why is `POP {R4–R6, PC}` useful?

It restores callee-saved registers and returns in one instruction, especially useful after `PUSH {R4–R6, LR}`.

3. What does partial decoding buy you?

Simpler, smaller, often faster decoder logic, at the cost of mirrored address regions.

4. Why does `ADC` matter after `ADD`?

It consumes the carry-out from the low word so you can continue the addition into the next more significant word.

FINAL REVIEW CHECKLIST

- I can distinguish address formation from memory loading.
- I can compute all common ARM effective addresses.
- I know which load forms sign-extend and zero-extend.
- I can use condition codes correctly for signed and unsigned comparisons.
- I can read and build IT blocks.
- I can do 64-bit or 96-bit add/sub across 32-bit registers.
- I know how `BL`, `LR`, and `BX LR` work together.
- I can save and restore register groups with PUSH/POP or STM/LDM.
- I know the AAPCS roles of R0-R3, R4-R11, LR, and SP.
- I can trace SP through a subroutine.
- I can explain the difference between pass-by-value and pass-by-reference.
- I can explain active-low, asserted, negated, and tri-state.
- I can determine number of transfers from data-bus width vs memory-port width.
- I can read setup, hold, and propagation timing from a diagram.
- I can explain DMA and its modes.
- I can design full or partial address decoders from base address and size.

Prepared from OCR text in the current folder on 2026-03-31. This is a study aid, not a replacement for the official course notes or ARM documentation.